

Below is the English translation, with the original structure, headings, bullet points, and layout preserved. The uploaded document is the source.

Capstone Project — Laravel Blog

Design Exercise · Individual Assignment

1. Context

As part of the DWWM training program, you will develop a blog from A to Z using Laravel. This capstone project runs throughout the backend sequence: every concept covered in class—routing, Blade, Eloquent, authentication, middleware—will be applied directly to this project.

Before writing a single line of code, you must design what you are going to build. A developer who codes without first modeling the application accumulates inconsistencies, backtracks frequently, and wastes time.

This exercise forces you to think first.

The deliverables you produce here will serve as your reference throughout development. Keep them accessible.

2. Project Description

The blog is a web application with three types of actors:

- A non-authenticated visitor can browse and read the blog freely without an account.
 - A registered user can comment on articles and manage their own comments.
 - An administrator manages all content: articles, categories, tags, and comments.
-

3. Specifications

Visitor (Not Logged In)

- View the paginated list of published articles (from newest to oldest)
- Filter the list by category
- Filter the list by tag
- Combine category and tag filters simultaneously
- View the details of an article: title, content, category, tags, comments
- Read comments on an article

User (Logged In)

- Register with a name, email address, and password
- Log in / log out
- Post a comment on an article
- Edit their own comment
- Delete their own comment

Administrator

- Log in / log out
- Create an article (status: draft or published)
- Edit an article
- Publish or unpublish an article
- Delete an article
- Delete any comment
- Create, edit, and delete categories
- Create, edit, and delete tags

Business Rules

- Only articles with the "**published**" status are visible to visitors and users. Drafts remain invisible outside the administration interface.
 - An article belongs to exactly one category and may have multiple tags.
 - Comments are visible to everyone—logged in or not.
 - Posting, editing, or deleting a comment requires being logged in.
 - A user may only edit or delete their own comments.
 - The article list is paginated (using the framework's default pagination value) and sorted from newest to oldest.
-

4. Expected Deliverables

Produce the following three deliverables.

The format is flexible—hand-drawn, online tools, etc.—as long as it is readable and exportable as a PDF or image.

Deliverable	Expected Content	Suggested Tools
Use Cases	List of use cases by actor. For each use case: name, actor, trigger, expected outcome.	Word/PDF table, draw.io, Excalidraw, scanned paper

Deliverable	Expected Content	Suggested Tools
Wireframes	Wireframes of the main screens: article list, article details, comment form, article creation/edit form (admin). No visual design required—blocks and areas are sufficient.	Excalidraw, Figma, Balsamiq, scanned paper
Database Modeling	Conceptual Data Model (CDM/MCD) then Logical Data Model (LDM/MLD): all entities, their attributes with data types, relationships, and cardinalities. The LDM must be ready to be translated directly into Laravel migrations.	Looping, draw.io, dbdiagram.io, scanned paper

Tip: What Order Should You Work In?

1. Start with the use cases—they force you to list all possible actions.
2. The actions reveal the entities you need (Article, User, Comment, etc.).
3. The entities and their relationships give you the CDM (MCD), then the LDM (MLD).
4. The LDM shows which data is available—you can then create wireframes knowing what information should be displayed.

In this order, each deliverable naturally follows from the previous one.

5. Out of Scope

The following features are **not part of the project**. Do not model them and do not develop them.

- Image or media uploads for articles
- Notification system
- Full-text search engine
- Nested comment replies (threaded discussions)

 These features are natural and realistic extensions of the project. If you want to explore them outside of class sessions, feel free to do so—but the training program's scope ends with the specifications above.

6. Submission

Your deliverables must be submitted before you begin coding.

You will then receive a reference correction to ensure your modeling is consistent before implementation.

Repository Organization

You will create a dedicated GitHub repository for this project: **DWWM-blog**.

This repository will host both your Laravel code and your design documents.

Create a **docs/** folder at the root level to store your deliverables:

```
DWWM-blog/
├── docs/
│   ├── use-cases.pdf      ← or .png, .md, any format
│   ├── wireframes.pdf
│   └── modelisation.pdf
├── app/                  ← Laravel code (later)
└── ...
```

Expected Git Practices

Work with Git from the design phase—not only when coding.

Apply the following rules:

- Commit frequently: make a commit whenever you complete an identifiable piece of work (a use case, a screen, the CDM, etc.).
- Atomic commits: one commit = one change. Do not combine "added CDM" and "fixed use case 3" in the same commit.
- Explicit commit messages: the message should describe what was done, not simply "update" or "modification." Use the conventional prefixes learned in class.

💡 Why Commit Design Documents?

Your Git history tells the story of the evolution of your thinking, not just your code.

If your modeling changes during development, you retain a record of the previous version. This is also how professional teams work: design documents live in the repository, not forgotten in a corner of someone's desk.

⚠️ Important

An inconsistent model will impact all the code that follows.

It is better to correct modeling issues before moving on to implementation.

If you are unsure about any point in the specifications, ask questions before creating your model.